

UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE

Nombre: Josue Chango

Materia: Investigación en ingeniería de software

Tema: Técnicas de investigación

2026

Introducción

La Ingeniería de Software es una disciplina que no puede avanzar solo con intuición o experiencia acumulada: necesita métodos que generen conocimiento sólido y verificable. Las técnicas de investigación científica cumplen ese rol; son las herramientas que los investigadores usan para estudiar fenómenos, probar hipótesis y construir teoría en torno al desarrollo y la gestión de software.

1. Descripción de las Técnicas de Investigación

Las ocho técnicas que se describen a continuación representan el repertorio metodológico más utilizado en la investigación empírica de la Ingeniería de Software. Cada una parte de supuestos distintos sobre qué cuenta como evidencia y cómo obtenerla

1.1 Experimento Controlado

El experimento controlado es la técnica más cercana al ideal del método científico clásico. El investigador manipula deliberadamente una o más variables (las causas que quiere estudiar) y mide el efecto que eso produce en otras variables, mientras mantiene todo lo demás constante. Esa combinación de manipulación y control es lo que hace posible afirmar que A causó B, no solo que A y B aparecieron juntos.

Características principales:

- El investigador manipula las variables independientes de forma deliberada y planificada.
- El entorno se controla para que factores externos no contaminen los resultados.
- Los participantes se asignan aleatoriamente a grupos, lo que garantiza que los grupos sean comparables desde el inicio.
- Es la única técnica que permite establecer relaciones causales con alta confianza estadística.
- Los resultados son atribuibles a lo que se manipuló, no a variables escondidas.
- Otro investigador puede repetir el experimento y obtener resultados similares.
- Las métricas son objetivas y cuantificables.

Particularidades en Ingeniería de Software:

- Se usa para comparar metodologías de desarrollo, por ejemplo TDD frente al desarrollo sin pruebas previas.
- También se evalúan herramientas, lenguajes de programación o patrones de diseño bajo condiciones equiparables.
- Los participantes suelen ser desarrolladores profesionales o estudiantes avanzados de ingeniería.
- Las métricas más frecuentes incluyen tiempo de desarrollo, densidad de defectos y productividad medida.

1.2 Quasi-Experimento

Cuando no es posible asignar participantes aleatoriamente a los grupos, se recurre al quasi-experimento. La diferencia con el experimento controlado no es menor: sin aleatorización, los grupos pueden ser distintos desde el principio, lo que complica la interpretación de los resultados. Aun así, el investigador sí manipula la variable de interés, y eso lo distingue de los estudios meramente observacionales.

Características principales:

- No hay asignación aleatoria; los grupos se forman por circunstancias previas al estudio.
- El investigador sí manipula la variable independiente, aunque con menos control que en el experimento puro.
- Las variables externas son más difíciles de controlar, lo que reduce la validez interna.
- Se compensan esas limitaciones con técnicas estadísticas diseñadas para grupos no equivalentes.
- Su validez interna es inferior al experimento, pero supera ampliamente a los estudios solo observacionales.

Particularidades en Ingeniería de Software:

- Comparación de equipos de desarrollo preexistentes dentro de una organización.
- Evaluación de herramientas en proyectos reales donde aleatorizar sería inviable o disruptivo.
- Estudios en entornos industriales donde el investigador no puede decidir quién hace qué.

1.3 Estudio de Caso

El estudio de caso investiga un fenómeno en profundidad dentro de su contexto real. No busca medir variables bajo condiciones controladas sino entender cómo y por qué ocurren las cosas en proyectos o procesos de software concretos. Para eso combina varias fuentes de evidencia: entrevistas, documentos internos, observación directa, artefactos del proyecto.

Características principales:

- Foco en uno o varios casos específicos: un proyecto, un equipo, una organización.
- Triangula datos de múltiples fuentes para construir una imagen más completa y confiable del fenómeno.
- Trabaja en el contexto natural del objeto de estudio, sin crear condiciones artificiales.
- Puede ser exploratorio (¿qué pasa aquí?), descriptivo (¿cómo ocurre?) o explicativo (¿por qué?).
- Produce hallazgos ricos en detalle, aunque no se pueden generalizar estadísticamente a toda una población.

Particularidades en Ingeniería de Software:

- Análisis de proyectos exitosos o fallidos para extraer lecciones aprendidas aplicables a futuras iniciativas.
- Evaluación de cómo una empresa específica adoptó metodologías ágiles y qué obstáculos encontró.

- Estudio de prácticas de revisión de código, gestión de deuda técnica o implementación de DevOps en contexto real.

1.4 Encuesta (Survey)

La encuesta recoge datos de una muestra amplia de personas mediante cuestionarios. Su mayor ventaja es el alcance: permite conocer opiniones, prácticas o percepciones de cientos o miles de profesionales de manera relativamente rápida y con un costo moderado. Lo que no puede hacer es explicar causas; solo describe o muestra correlaciones.

Características principales:

- Datos recogidos mediante cuestionarios, que pueden ser totalmente cerrados (escala numérica) o incluir preguntas abiertas.
- Eficiente para estudiar poblaciones grandes que sería imposible observar directamente.
- El análisis estadístico identifica patrones, tendencias y relaciones entre variables.
- Si la muestra es aleatoria y representativa, los resultados son generalizables a la población de origen.
- Tiene un costo menor que los experimentos o los estudios de caso de larga duración.

Particularidades en Ingeniería de Software:

- Estudios sobre las prácticas de desarrollo más extendidas en la industria, como el Stack Overflow Developer Survey.
- Medición de la satisfacción de desarrolladores con herramientas, frameworks o metodologías.
- Identificación de habilidades más buscadas o problemas más frecuentes en equipos de software.
- Seguimiento de la adopción de tecnologías emergentes como cloud, inteligencia artificial o arquitecturas de microservicios.

1.5 Investigación-Acción (Action Research)

En la investigación-acción el investigador no observa desde afuera: entra al contexto, trabaja junto a los practicantes para resolver un problema real y, mientras lo hace, genera conocimiento científico. El proceso es cíclico: se planifica una intervención, se ejecuta, se observa qué ocurrió, se reflexiona sobre ello y se ajusta el plan para el siguiente ciclo.

Características principales:

- El investigador es parte activa del proceso, no un observador externo.
- Estructura en ciclos iterativos: planificación → acción → observación → reflexión → nueva planificación.
- El objetivo es doble: resolver un problema concreto y producir conocimiento transferible.
- Los resultados se aplican de inmediato al contexto donde surgieron, lo que le da una validez ecológica muy alta.
- Requiere colaboración constante con los profesionales del campo estudiado.

Particularidades en Ingeniería de Software:

- Implementación de mejoras en procesos de desarrollo dentro de una empresa, evaluando el impacto en tiempo real.
- Introducción de prácticas ágiles y ajuste iterativo según los problemas que van apareciendo.
- Proyectos de mejora de calidad de software donde el equipo de desarrollo participa activamente en el diseño de las soluciones.

1.6 Revisión Sistemática de Literatura (SLR)

La revisión sistemática no produce datos primarios: sintetiza lo que ya se ha investigado. Su valor está en la rigurosidad del proceso, que sigue un protocolo predefinido con criterios explícitos de búsqueda, selección y análisis. Eso la distingue de una revisión narrativa informal y la hace replicable y objetiva.

Características principales:

- Protocolo definido antes de iniciar la búsqueda, con criterios claros de inclusión y exclusión de estudios.
- Búsqueda exhaustiva en múltiples bases de datos científicas: IEEE Xplore, ACM, Scopus, entre otras.
- Cada estudio candidato pasa por una evaluación de calidad antes de ser incluido.
- Los resultados se sintetizan cuantitativamente (meta-análisis) o de forma cualitativa, según la naturaleza de los estudios encontrados.
- El proceso es transparente y replicable: otro investigador que siga el mismo protocolo debería llegar a resultados similares.

Particularidades en Ingeniería de Software:

- Comparación de la efectividad de metodologías de desarrollo en decenas de estudios primarios.
- Revisión del estado del arte en áreas como pruebas de software, métricas de calidad o técnicas de estimación de esfuerzo.
- Identificación de vacíos de conocimiento que justifican nuevas investigaciones primarias.

1.7 Investigación Etnográfica

La etnografía viene de la antropología. El investigador se integra durante un período prolongado al grupo que estudia, participa de sus rutinas y registra lo que observa con detalle. El objetivo no es medir sino comprender desde adentro: qué valores guían al equipo, cómo toman decisiones, qué significado le dan a sus prácticas cotidianas.

Características principales:

- El investigador se sumerge en el entorno natural del grupo durante semanas o meses.
- La observación es participante: el investigador convive con el grupo, no solo lo mira.
- Los datos son cualitativos: notas de campo, conversaciones informales, artefactos del trabajo diario.
- Busca entender los significados y valores del grupo desde su propia perspectiva.

- Las teorías emergen de los datos, no se prueban hipótesis definidas a priori.

Particularidades en Ingeniería de Software:

- Comprensión de la cultura de un equipo de desarrollo: cómo colaboran, cómo gestionan el conflicto, qué normas tácitas rigen su trabajo.
- Análisis de cómo los desarrolladores resuelven problemas técnicos difíciles en situaciones reales, con presiones y recursos limitados.
- Estudio de la comunicación informal y la toma de decisiones en equipos ágiles distribuidos.

1.8 Simulación y Modelado

La simulación construye un modelo formal de un sistema o proceso y lo ejecuta para estudiar su comportamiento en distintos escenarios. En lugar de tocar el sistema real, el investigador experimenta sobre la representación abstracta, lo que elimina riesgos y reduce costos. La clave está en que el modelo sea suficientemente fiel a la realidad.

Características principales:

- Se construye un modelo abstracto que captura las reglas y relaciones esenciales del sistema real.
- Se pueden probar decenas de escenarios distintos en el tiempo que llevaría un solo experimento real.
- Maneja sistemas con muchas variables interdependientes que serían imposibles de controlar en un experimento físico.
- El modelo necesita validarse contra datos reales para confirmar que sus predicciones son creíbles.
- Los tipos más usados en IS son: simulación discreta, continua, basada en agentes y Monte Carlo.

Particularidades en Ingeniería de Software:

- Modelado de procesos de desarrollo para predecir tiempos de entrega y costos antes de iniciar un proyecto.
- Simulación del comportamiento de un sistema bajo distintas cargas de trabajo, para detectar cuellos de botella sin arriesgar el sistema en producción.
- Modelos de gestión de riesgos y estimación de esfuerzo en proyectos con alta incertidumbre.

2. Uso de las Técnicas: Cuándo y Por Qué

Elegir bien la técnica es tan importante como aplicarla correctamente. La selección depende, sobre todo, del tipo de pregunta que se quiere responder, del acceso que el investigador tiene al contexto y de los recursos disponibles. La siguiente tabla resume esa decisión para cada técnica.

Técnica	Cuándo Utilizarla	Circunstancias Favorables
Experimento Controlado	Cuando se necesita probar si A causa B con rigor científico	Laboratorio o ambiente controlado, participantes disponibles para asignación aleatoria
Quasi-Experimento	Cuando no es posible aleatorizar, pero sí manipular la variable de interés	Entornos industriales con equipos preexistentes, restricciones éticas o prácticas
Estudio de Caso	Para entender en profundidad un fenómeno complejo en su contexto real	Proyectos únicos, fenómenos nuevos, cuando importa más la profundidad que el alcance
Encuesta	Para conocer opiniones o prácticas de una población amplia	Acceso a muchos profesionales, estudios descriptivos o de prevalencia
Investigación-Acción	Cuando se quiere resolver un problema real y generar conocimiento al mismo tiempo	Colaboración activa con la industria, mejora de procesos organizacionales
Revisión Sistemática	Para saber qué se ha investigado ya sobre un tema antes de iniciar un estudio primario	Literatura publicada abundante, preguntas sobre el estado del arte
Etnografía	Para comprender la cultura y el comportamiento cotidiano de un equipo	Acceso prolongado al entorno, fenómenos sociales o culturales en IS
Simulación	Para modelar y predecir el comportamiento de sistemas complejos sin riesgo real	Experimentación real demasiado costosa o riesgosa, sistemas aún no construidos

3. Estudio Comparativo de las Técnicas

Comparar estas técnicas no implica buscar cuál es la mejor. Cada una tiene un propósito diferente y responde bien a un tipo específico de pregunta. El análisis que sigue las confronta en dimensiones que sí son comparables directamente: el paradigma del que parten, el nivel de control que ofrecen, el costo y tiempo que exigen, y su balance de fortalezas y debilidades.

3.1 Comparación por Paradigma y Enfoque

Técnica	Paradigma	Enfoque	Nivel de Control	Generalización
Experimento Controlado	Positivista	Cuantitativo	Muy Alto	Alta (estadística)
Quasi-Experimento	Positivista	Cuantitativo	Alto	Moderada
Estudio de Caso	Interpretativo	Cualitativo/Mixto	Bajo	Analítica
Encuesta	Positivista	Cuantitativo	Moderado	Alta (si aleatoria)
Investigación-Acción	Crítico	Mixto	Moderado	Baja

Técnica	Paradigma	Enfoque	Nivel de Control	Generalización
Revisión Sistemática	Positivista	Mixto	Alto (protocolo)	Alta (síntesis)
Etnografía	Interpretativo	Cualitativo	Muy Bajo	Muy Baja
Simulación	Positivista	Cuantitativo	Muy Alto	Alta (modelos)

3.2 Comparación por Recursos y Tiempo

Técnica	Costo	Tiempo	Complejidad	Replicabilidad
Experimento Controlado	Alto	Moderado	Alta	Alta
Quasi-Experimento	Moderado	Moderado	Media-Alta	Moderada
Estudio de Caso	Moderado	Largo	Media	Baja
Encuesta	Bajo-Moderado	Corto-Moderado	Baja-Media	Alta
Investigación-Acción	Moderado	Largo	Media	Baja
Revisión Sistemática	Bajo	Largo	Alta	Muy Alta
Etnografía	Alto	Muy Largo	Alta	Baja
Simulación	Variable	Moderado	Alta	Alta

3.3 Fortalezas y Debilidades

Técnica	Fortalezas	Debilidades
Experimento Controlado	Alta validez interna; permite inferir causalidad; fácilmente replicable	Baja validez externa; entorno artificial; difícil de ejecutar con profesionales reales
Quasi-Experimento	Más cercano a la realidad que el experimento puro; causalidad parcialmente inferible	Sesgos por falta de aleatorización; conclusiones causales más débiles
Estudio de Caso	Profundidad de análisis; contexto real; múltiples fuentes de evidencia	Generalización estadística limitada; mayor riesgo de sesgo del investigador
Encuesta	Alcance amplio; eficiente; datos comparables entre estudios	Sesgo de respuesta; poca profundidad; no establece causalidad
Investigación-Acción	Impacto práctico inmediato; colaborativa; válida en el contexto de origen	Objetividad reducida por el rol dual del investigador; baja replicabilidad
Revisión Sistemática	Sintetiza gran volumen de evidencia; objetiva; detecta vacíos de conocimiento	No genera datos propios; depende de la calidad de los estudios existentes

Técnica	Fortalezas	Debilidades
Etnografía	Comprensión profunda de cultura y comportamiento; alta validez ecológica	Muy costosa en tiempo; sesgo del observador; prácticamente irreplicable
Simulación	Segura, económica y permite explorar muchos escenarios sin riesgo	Los resultados dependen de qué tan fiel sea el modelo; requiere validación rigurosa

4. Conclusiones y Recomendaciones

4.1 Conclusiones

1. La IS necesita métodos cuantitativos y cualitativos por igual. No es una disciplina que pueda contentarse con uno solo. Los experimentos miden con precisión, pero los estudios de caso y la etnografía explican por qué las cosas ocurren como ocurren. Prescindir de alguno empobrece el conocimiento.
2. No existe una técnica que sirva para todo. La pregunta de investigación es la que manda. Si se quiere saber si A causa B, hace falta un experimento. Si se quiere entender cómo trabaja un equipo real, hay que ir al campo. Elegir la técnica antes de formular bien la pregunta es un error frecuente.
3. El rigor no es opcional. Sea cual sea la técnica, la calidad de la investigación depende de aplicarla con disciplina, documentar el proceso con transparencia y reportar honestamente sus limitaciones.
4. El experimento sigue siendo el estándar de oro para la causalidad. Cuando el objetivo central es probar que una variable causa cambios en otra, ninguna otra técnica lo hace con la misma solidez. Su talón de Aquiles es la validez externa: lo que pasa en el laboratorio no siempre se traslada a un proyecto industrial real.
5. La revisión sistemática es el punto de partida obligatorio. Antes de diseñar cualquier estudio primario, conviene saber qué se conoce ya. La revisión sistemática evita duplicar esfuerzos y deja claro qué vacíos justifican investigar de nuevo.
6. La triangulación metodológica es la tendencia dominante. Los estudios actuales en IS tienden a combinar técnicas para compensar sus debilidades individuales. Una encuesta amplia más un estudio de caso profundo dan mucho más que cualquiera de los dos solos.

4.2 Recomendaciones

1. Definir la pregunta de investigación antes de pensar en la técnica. Una pregunta bien formulada señala casi directamente el método apropiado: '¿Qué causa X?' apunta al experimento; '¿Cómo ocurre X en la práctica?' apunta al estudio de caso o la etnografía.
2. Para proyectos aplicados en la industria, optar por investigación-acción o estudios de caso. Ambos se adaptan a proyectos y equipos reales sin exigir condiciones de laboratorio ni interrumpir los procesos de la organización.
3. En estudios académicos sobre efectividad de herramientas o metodologías, priorizar los experimentos o quasi-experimentos. Son los únicos que permiten extraer conclusiones causales con respaldo estadístico.
4. Tratar la revisión sistemática como un paso previo obligatorio, no opcional. Fundamentar la relevancia del trabajo sobre lo que ya existe en la literatura es una exigencia científica básica.
5. Cuando experimentar en el sistema real sea demasiado costoso o riesgoso, la simulación es una alternativa legítima y rigurosa, siempre que el modelo se valide contra datos empíricos.
6. Si los recursos lo permiten y el problema es complejo, diseñar estudios con triangulación. Combinar una encuesta de alcance amplio con estudios de caso selectivos ofrece una comprensión que ningún método aislado puede dar.
7. Reportar siempre las amenazas a la validez del estudio: interna, externa, de constructo y estadística. No hacerlo no significa que esas amenazas no existan; solo significa que no se examinaron con honestidad.

Referencias Bibliográficas

- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer.
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131-164.
- Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering*. Technical Report EBSE-2007-01, Keele University.
- Creswell, J. W. (2014). *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (4th ed.). SAGE Publications.
- Easterbrook, S., Singer, J., Storey, M. A., & Damian, D. (2008). Selecting empirical methods for software engineering research. In *Guide to Advanced Empirical Software Engineering* (pp. 285-311). Springer.
- Jedlitschka, A., Ciolkowski, M., & Pfahl, D. (2008). Reporting experiments in software engineering. In *Guide to Advanced Empirical Software Engineering* (pp. 201-228). Springer.

Dingsøyr, T., Nerur, S., Balijepally, V., & Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6), 1213-1221.